

 TECHNICAL DOCUMENTATION

AgriPlatform

Backend Technical Report

A comprehensive analysis of the Agriculture Profit-Sharing Platform backend — architecture, modules, APIs, blockchain integration, and infrastructure.

PROJECT	API VERSION	FRAMEWORK	DATE
agri-platform	v1.0	NestJS + TypeScript	September 2026
REPORT TYPE			
Architecture & API			

 NAVIGATION

Table of Contents

Complete guide to all backend components and sections in this report.

01	Project Overview	§1
02	Technology Stack	§2
03	Architecture & Infrastructure	§3
04	Backend Modules (25 Modules)	§4
05	Authentication System	§5
06	Blockchain Integration	§6
07	Database Layer	§7
08	Smart Contracts	§8
09	REST API Endpoints	§9
10	Rust Indexer Service	§10
11	Security & Middleware	§11
12	Infrastructure & Docker	§12

§ 01

Project Overview

AgriPlatform is a full-stack, blockchain-powered agriculture profit-sharing platform that connects farmers and investors through smart contracts, escrow management, and transparent on-chain settlement.

Platform Purpose

The platform digitizes agricultural profit-sharing agreements between **Farmers**, **Investors**, and **Admins**. Farmers list agricultural projects and deals; Investors fund them; profits are distributed automatically through on-chain smart contracts.



Farmer

Creates farm profiles, projects, crop listings, and deals.
Receives profit disbursements.



Investor

Browses deals, makes investments, monitors returns, and manages wallet & escrow.



Admin

Reviews & approves deals, monitors platform stats, manages users, and audits activity.



Blockchain

All financial agreements are enforced via Solidity smart contracts on EVM chains.

Monorepo Structure

```
agri-platform/
├── apps/
│   ├── api/           # NestJS backend (Node.js + TypeScript)
│   └── web/           # Next.js frontend
├── contracts/         # Solidity smart contracts (Foundry)
├── crates/
│   └── gram-indexer/  # Rust blockchain event indexer
├── packages/          # Shared TypeScript packages
└── docker-compose.yml # Full infra stack
```

§ 02

Technology Stack

Complete list of technologies, frameworks, and tools used across the backend system.

Category	Technology	Version / Detail	Purpose
Runtime	Node.js	LTS	JavaScript runtime for the API server
Language	TypeScript	Strict mode	Type-safe backend development
Framework	NestJS	v10+	Modular server-side application framework
ORM	Prisma	Latest	Database access and schema management
Database	PostgreSQL	16-alpine	Primary relational database
Cache	Redis	7-alpine	Caching, rate limiting, session store
Object Store	MinIO	Latest	S3-compatible file/media storage
Blockchain Client	viem	^2.56.5	EVM chain interaction (read/write)
Smart Contracts	Solidity + Foundry	—	On-chain deal and escrow logic
Indexer	Rust (gram-indexer)	Tokio async	High-performance blockchain event indexer
Auth	JWT + Web3 Signatures	—	Traditional + wallet-based authentication
API Docs	Swagger / OpenAPI	@nestjs/swagger	Auto-generated API documentation
Validation	class-validator	—	DTO validation with decorators
Events	@nestjs/event-emitter	—	Internal domain event bus
Security	Helmet + Compression	—	HTTP security headers and GZIP
Monorepo	Turborepo + npm workspaces	^2.4.4	Build orchestration and caching
Containerization	Docker Compose	v3.9	Local development infrastructure

Category	Technology	Version / Detail	Purpose
x402 Protocol	@x402/core, @x402/evm	^2.25.0	Crypto payment protocol integration

§ 03

Architecture & Infrastructure

The backend follows a clean modular architecture using NestJS's dependency injection container, with a layered approach: Controller → Service → Repository → Database.

Request Lifecycle



API Configuration

Setting	Value	Notes
Port	3001	Configurable via <code>PORT</code> env var
API Prefix	<code>/api/v1</code>	All routes prefixed
Swagger UI	<code>/api/docs</code>	Toggleable via <code>SWAGGER_ENABLED</code>
CORS	Configurable origins	Default: <code>http://localhost:3000</code>
Validation	Whitelist + Transform	Global <code>ValidationPipe</code>
Auth	Bearer JWT	Header: <code>Authorization: Bearer <token></code>

Global Middlewares & Pipes



Applied to Every Request

- Helmet** — Adds 11 security HTTP headers (XSS, CSP, HSTS, etc.)
- Compression** — GZIP compression for all responses

- **CORS** — Allowlist-based cross-origin policy with credentials
- **ValidationPipe** — Auto-validates DTOs, strips unknown fields, transforms types
- **Shutdown Hooks** — Graceful shutdown via `app.enableShutdownHooks()`

§ 04

Backend Modules (25 Total)

The NestJS application is broken into 25 focused modules, each responsible for a distinct domain. All are registered in `AppModule`.

Core Infrastructure Modules

 **DatabaseModule**

 **ConfigModule**

 **EventEmitterModule**

 **CommonModule**

Domain Business Modules

 **AuthModule**

 **UsersModule**

 **FarmersModule**

 **InvestorsModule**

 **FarmsModule**

 **CropsModule**

 **ProjectsModule**

 **DealsModule**

Financial Modules

 **InvestmentsModule**

 **PaymentsModule**

 **EscrowModule**

 **SettlementsModule**

 **ProfitsModule**

 **LedgerModule**

 **WalletsModule**

Platform & Utility Modules

 **BlockchainModule**

 **DashboardModule**

 **AdminModule**

 **AuditModule**

 **NotificationsModule**

 **OracleModule**

 **WebhooksModule**

Module Responsibilities

Module	Key Responsibility	Notable Features
DealsModule	Full deal lifecycle management	Submit, review, approve, reject, fund, settle
EscrowModule	Escrow account management	Lock/release funds, blockchain contract address tracking
ProfitsModule	Profit calculation & distribution	Smart contract-triggered disbursements
LedgerModule	Immutable financial ledger	Double-entry accounting records
AuditModule	Admin action audit trail	Actor email, role, old/new values logged
OracleModule	External price/data feeds	Crop price oracle integration
DashboardModule	Aggregated analytics	Stats, revenue series, investment breakdown, activity
WebhooksModule	External webhook delivery	Event-driven notifications to third parties
BlockchainModule	EVM chain interaction	viem public/wallet clients, multi-chain support

§ 05

Authentication System

The platform supports dual authentication — traditional JWT-based email/password and Web3 wallet signature authentication for blockchain-native users.

Auth Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/v1/auth/register	Register new user account	No
POST	/api/v1/auth/login	Login with email + password	No
POST	/api/v1/auth/refresh	Refresh JWT access token	No
GET	/api/v1/auth/me	Get current authenticated user	✔ JWT
POST	/api/v1/auth/logout	Invalidate current session	✔ JWT
POST	/api/v1/auth/web3/nonce	Generate nonce for Web3 auth	No
POST	/api/v1/auth/web3/login	Login via wallet signature	No
POST	/api/v1/auth/forgot-password	Request password reset email	No
POST	/api/v1/auth/reset-password	Reset password with token	No

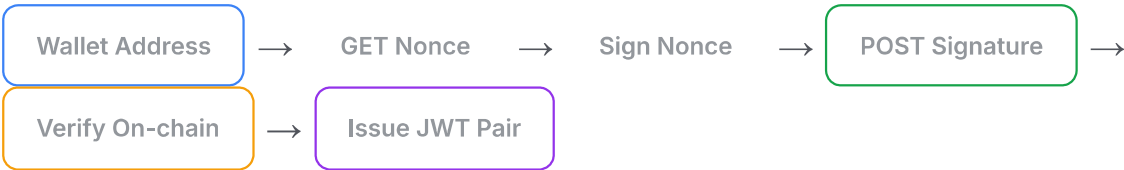
Token System

🔑

JWT Token Pair

- **accessToken** — Short-lived JWT for API authorization
- **refreshToken** — Long-lived token to get new access tokens
- Refresh tokens are **revocable** (stored and invalidated on logout)
- Passwords are **never returned** — sanitized via `sanitizeUser()`

Web3 Authentication Flow



User Roles

Role	Access Level	Capabilities
ADMIN	Platform-wide	Deal approval, user management, full audit access, dashboard overview

Role	Access Level	Capabilities
FARMER	Own resources	Create farms, projects, deals; receive payments; view own stats
INVESTOR	Own resources	Browse deals, invest funds, manage wallet, view portfolio

Security Guards

 Guard Stack

- **JwtAuthGuard** — Validates Bearer token on protected routes
- **RolesGuard** — Role-based access control via `@Roles()` decorator
- **CurrentUser decorator** — Injects authenticated user into controller methods

§ 06

Blockchain Integration

The `BlockchainService` is the core EVM interface, using the **viem** library to interact with smart contracts on Hardhat (local), Sepolia (testnet), and Mainnet.

Supported Networks

Network	Chain ID	Use Case
Hardhat Local	31337	Local development and testing
Sepolia Testnet	11155111	Staging & QA environment
Ethereum Mainnet	1	Production deployment

BlockchainService Methods

Method	Return Type	Description
<code>getBlockNumber()</code>	<code>bigint</code>	Current chain block height

Method	Return Type	Description
<code>getTransaction(hash)</code>	Transaction	Fetch transaction by hash
<code>getTransactionReceipt(hash)</code>	Receipt null	Get confirmed receipt
<code>waitForConfirmation(hash, n)</code>	Receipt	Wait for N block confirmations (120s timeout)
<code>estimateGas(params)</code>	<code>bigint</code>	Estimate gas cost before sending
<code>sendTransaction(params)</code>	Hash	Sign & broadcast transaction via wallet client
<code>getContractEventLogs()</code>	Log[]	Fetch contract events in block range
<code>getBalance(address)</code>	<code>bigint</code>	Get ETH balance of an address
<code>getPublicClient()</code>	PublicClient	Read-only viem client
<code>getWalletClient()</code>	WalletClient	Signing wallet client (requires private key)

Environment Configuration

```
BLOCKCHAIN_CHAIN_ID=31337           # 1=mainnet, 11155111=sepolia
BLOCKCHAIN_RPC_URL=http://127.0.0.1:8545
BLOCKCHAIN_PRIVATE_KEY=0x...        # Platform signer private key
BLOCKCHAIN_CONFIRMATIONS=1          # Blocks to wait for finality
```

⚡ Transport Config

- HTTP timeout: **2,000ms** with 1 retry
- Transaction wait timeout: **120 seconds**
- Falls back gracefully if RPC is offline (returns `0n` for block number)
- Wallet client only initialized when `BLOCKCHAIN_PRIVATE_KEY` is set

Database Layer

PostgreSQL 16 is the primary database, accessed through Prisma ORM. The `PrismaService` extends `PrismaClient` and implements graceful connection management with timeout handling.

PrismaService Features

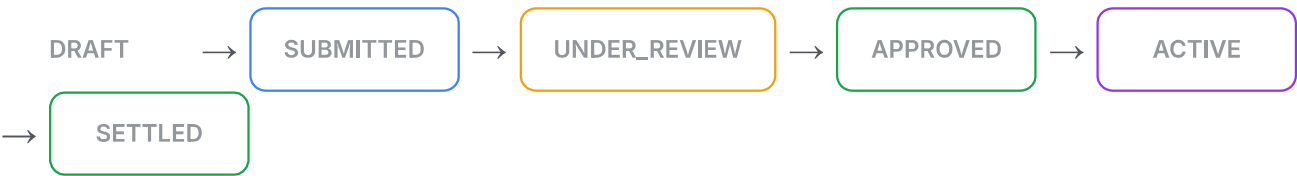
 Database Service Capabilities

- **Graceful startup** — 2-second connection timeout with development fallback
- **Query logging** — All queries emitted as events for monitoring
- **Graceful shutdown** — Disconnects cleanly on module destroy
- **Test cleanup** — `cleanDatabase()` method for test isolation
- **Production guard** — Throws on connection failure in `NODE_ENV=production`

Key Data Models

Model	Key Fields	Relations
User	id, email, passwordHash, role, isActive, lastLoginAt	FarmerProfile, InvestorProfile, Notifications, AuditLogs
Deal	id, title, status, fundingTarget, farmerProfileId	Project, DealTerm, EscrowAccount, Investments, Participants
EscrowAccount	id, status, balance, lockedBalance, blockchainContractAddress	Deal, Transactions
AuditLog	id, actorId, actorEmail, action, entityType, oldValues, newValues	User (actor)
Notification	id, userId, type, title, message, isRead, data	User
FarmerProfile	id, userId, farmName, verificationStatus	User, Farms, Deals
InvestorProfile	id, userId, totalInvested	User, Investments, DealParticipants

Deal Status Lifecycle



Database Connection

```
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/agri_platform"
```

§ 08

Smart Contracts

Seven Solidity smart contracts are compiled with **Foundry** and deployed on EVM-compatible chains. They enforce all financial agreements on-chain without central authority.

Contract	File	Role
AgriPlatform	AgriPlatform.sol	Main platform registry, access control, admin management (3.5 KB)
DealContract	DealContract.sol	Individual deal terms, funding, settlement, profit logic (7.1 KB)
DealFactory	DealFactory.sol	Factory pattern — deploys new DealContract instances (5.1 KB)
Escrow	Escrow.sol	Funds lock/release, dispute resolution, withdrawal logic (2.9 KB)
Oracle	Oracle.sol	On-chain price feeds for agricultural commodities (4.3 KB)
ProfitDistribution	ProfitDistribution.sol	Automated profit calculation and investor payout (5.7 KB)
ProjectRegistry	ProjectRegistry.sol	On-chain project registration and metadata storage (2.2 KB)

Contract Architecture Pattern

- **Factory Pattern** — DealFactory deploys individual DealContract per deal
- **Escrow Pattern** — Funds held in Escrow until conditions are met
- **Oracle Pattern** — External price data injected for yield calculations
- **Registry Pattern** — AgriPlatform and ProjectRegistry as on-chain directories
- **Access Control** — access/ directory contains role-based modifiers

 Toolchain

- **Foundry** — Compile, test, and deploy Solidity contracts
- **foundry.toml** — Project configuration
- **contracts/test/** — Forge unit tests for all contracts
- **Hardhat node** — Local development blockchain at port 8545

§ 09

REST API Endpoints

All endpoints are prefixed with `/api/v1` . Full interactive documentation is available at `/api/docs` (Swagger UI).

Notifications API

Method	Endpoint	Description
GET	<code>/notifications</code>	Get paginated notifications for current user
GET	<code>/notifications/unread-count</code>	Get count of unread notifications
PATCH	<code>/notifications/:id/read</code>	Mark single notification as read
POST	<code>/notifications/:id/read</code>	Mark as read (POST alias)
POST	<code>/notifications/read-all</code>	Mark all notifications as read
DELETE	<code>/notifications/:id</code>	Delete a notification

Dashboard API

Method	Endpoint	Description
GET	<code>/dashboard/overview</code>	Unified dashboard overview (role-aware)
GET	<code>/dashboard/stats</code>	Aggregate platform or user statistics
GET	<code>/dashboard/activity</code>	Recent activity feed
GET	<code>/dashboard/revenue</code>	Monthly revenue time-series data

Method	Endpoint	Description
GET	/dashboard/investments/breakdown	Investment distribution by category

Deal Approval (Admin) API

Action	Description	Side Effects
Get Deals	Paginated list with filter by status, farmer, search	—
Get Deal Detail	Full deal with participants, project, escrow, crop info	—
Approve Deal	Admin approves submitted deal	AuditLog created, Farmer notified
Reject Deal	Admin rejects with reason	AuditLog created, Farmer notified with reason
Get Stats	Approval statistics summary	—

Other Module Endpoints (Summary)

Module	Base Path	Key Operations
Users	/users	CRUD, profile, role management
Farmers	/farmers	Profile, verification, farm listing
Investors	/investors	Profile, portfolio, investment history
Farms	/farms	Farm CRUD, location, area, crop association
Projects	/projects	Project lifecycle, revenue forecasting
Deals	/deals	Full deal lifecycle, funding, settlement
Investments	/investments	Investment creation, status, returns
Payments	/payments	Payment processing, history
Escrow	/escrow	Lock funds, release, balance query
Wallets	/wallets	Wallet management, transaction history

Module	Base Path	Key Operations
Ledger	/ledger	Financial entries, balance sheet
Profits	/profits	Profit records, distribution events
Settlements	/settlements	Deal settlement, payout confirmation
Audit	/audit	Audit log query, actor filtering
Crops	/crops	Crop types, pricing, season data

§ 10

Rust Indexer Service (gram-indexer)

A high-performance Rust microservice that indexes blockchain events from the deployed smart contracts into PostgreSQL, enabling the API to query historical on-chain data without slow RPC calls.

How It Works



Configuration

Environment Variable	Default	Description
DATABASE_URL	postgres://...	PostgreSQL connection string
REDIS_URL	redis://redis:6379	Redis for indexer state/caching
BLOCKCHAIN_RPC_URL	http://localhost:8545	EVM RPC endpoint to poll
AGRI_PLATFORM_ADDRESS	0x4F129B...	Main contract address to watch
INDEXER_POLL_INTERVAL_MS	1500	Block polling frequency
INDEXER_MAX_BLOCK_RANGE	2000	Max blocks per log query

Environment Variable	Default	Description
INDEXER_REORG_BUFFER_BLOCKS	12	Blocks held back for reorg safety
RUST_LOG	info,gram_indexer=debug	Log level configuration

 Why Rust for the Indexer?

- **Performance** — Handles thousands of events per second with minimal memory
- **Safety** — No runtime crashes, guaranteed memory safety
- **Async** — Tokio async runtime for non-blocking I/O
- **Reorg Handling** — 12-block buffer protects against chain reorganizations
- **Dockerized** — Runs as isolated container alongside the main stack

§ 11

Security & Middleware

Security is applied at multiple layers: HTTP headers, CORS, input validation, JWT guards, and role-based access control.

Security Layers

Layer	Implementation	What It Does
HTTP Headers	Helmet.js	XSS protection, HSTS, Content-Security-Policy, no-sniff
CORS	NestJS CORS	Origin allowlist, credentials support, method allowlist
Input Validation	ValidationPipe + class-validator	Rejects unknown fields, validates types, sanitizes input
Authentication	JwtAuthGuard (Passport JWT)	Validates signed JWT tokens on every protected route
Authorization	RolesGuard	Enforces RBAC (ADMIN / FARMER / INVESTOR)
Password Security	passwordHash field	Passwords stored as hashed values, never returned in API

Layer	Implementation	What It Does
Token Revocation	Refresh token tracking	Logout invalidates tokens server-side
Audit Trail	AuditModule	All admin actions logged with actor, IP, before/after values

⚠ Security Notes

- Set `SWAGGER_ENABLED=false` in production to disable the API docs page
- Use strong `JWT_SECRET` environment variable in production
- `BLOCKCHAIN_PRIVATE_KEY` must be kept secret — it can sign transactions
- CORS origins must be explicitly configured via `CORS_ORIGINS`

§ 12

Infrastructure & Docker

The full development stack runs in Docker Compose. Five services cover the database, cache, file storage, DB admin UI, and the Rust indexer.

Docker Compose Services

Service	Image	Port	Purpose
postgres	postgres:16-alpine	54322:5432	Primary PostgreSQL database
redis	redis:7-alpine	6380:6379	Cache and session store
minio	minio/minio:latest	9000, 9001	S3-compatible object storage + Console
pgadmin	dpage/pgadmin4	5050:80	PostgreSQL admin web UI
gram-indexer	Custom Rust build	—	Blockchain event indexer

Quick Start Commands

```
# Start all infrastructure
docker compose up -d
```

```
# Start API in development mode
npm run dev

# Build and run Rust indexer
npm run indexer:docker

# Access points:
# API:      http://localhost:3001
# Swagger:  http://localhost:3001/api/docs
# pgAdmin:  http://localhost:5050
# MinIO:    http://localhost:9001
```

Service Health Checks

- ✓ All Services Include Health Checks
- PostgreSQL — `pg_isready` every 10s, 5 retries
 - Redis — `redis-cli ping` every 10s, 5 retries
 - MinIO — `mc ready local` every 10s, 5 retries
 - gram-indexer — Waits for both PostgreSQL and Redis to be healthy

Data Volumes

Volume	Service	Purpose
postgres_data	PostgreSQL	Persistent database storage
redis_data	Redis	Persistent Redis AOF log
minio_data	MinIO	Persistent object/file storage
pgadmin_data	pgAdmin	pgAdmin configuration persistence